Números aleatórios, método de Monte Carlo e ajuste de parâmetros

30 de setembro de 2026

Números aleatórios, método de Monte Carlo e ajuste de parâmetros

30 de setembro de 2026

Números Pseudoaleatórios

- Sequências geradas por algoritmos determinísticos e reprodutíveis.
- Devem obedecer, no limite de muitas extrações, às propriedades estatísticas desejadas (IID).
- Em computadores, têm precisão finita (valores discretos).
- Métodos que usam repetidamente esses números são chamados de **Métodos de Monte Carlo**.

Geradores comuns:

- `lrand48` (C): $x_{n+1} = (ax_n + c) \bmod m$, período 2^{48}.
- `RANLUX`, L'Ecuyer, Mersenne-Twister (Sequência repete após $2^{19937} - 1$).

Gerador Uniforme com ROOT

```
// Exemplo: gerar numeros uniformes em [0,1) com TRandom3
void uniforme() {
    TRandom3 rng(12345); // semente
    TH1D *h = new TH1D("h", "Uniforme [0,1)", 100, 0, 1);
    for (int i = 0; i < 100000; i++) {
        double x = rng.Uniform(0, 1);
        h->Fill(x);
    }
    TCanvas *c = new TCanvas();
    h->Draw();
}
```

- `TRandom3` usa o algoritmo Mersenne-Twister.
- `Uniform(a,b)` remapeia linearmente para $[a, b)$.

Remapeamento de Números Uniformes

Dado x uniforme em $[0, 1[$, podemos obter x' uniforme em $[a, b[$:

$$x' = a + x(b - a)$$

- $x = 0 \Rightarrow x' = a$
- $x = 1 \Rightarrow x' = b$

Exemplo ROOT:

```
double x = rng.Uniform(0,1);
double xp = a + x*(b-a); // uniforme em [a,b)
```

Geradores Discretos

Para valores $1, \ldots, n$ com probabilidades $P_1, \ldots, P_n$, armazenamos probabilidades acumuladas:

$$C_k = \sum_{i=1}^{k} P_i, \quad k = 1, \ldots, n-1$$

Sorteia-se r uniforme em $[0, 1[$ e retorna-se o menor k tal que $C_k > r$.

```
// Exemplo: eficiencia de detector
double epsilon = 0.85;
double r = rng.Uniform(0,1);
bool detectado = (r < epsilon);
```

Método da Inversão da Cumulativa

Considere uma distribuição f da qual nós queremos obter números randômicos. Se $F(x) = \int_{-\infty}^{x} f(x')dx'$ e r uniforme em $[0, 1[$, então:

$$x = F^{-1}(r) \sim f(x)$$

Exemplo: distribuição exponencial

$$f(x) = \lambda e^{-\lambda x}, \quad F(x) = \int_0^x f(t)dt = 1 - e^{-\lambda x}$$

$$x = -\frac{1}{\lambda}\log(1 - r)$$

```
double lambda = 0.5;
double r = rng.Uniform(0,1);
double x = -log(1-r)/lambda;
```

Gerador Gaussiano: Box-Muller

Transformação de dois uniformes r_1, r_2 em dois normais padrão:

$$r = \sqrt{-2\log(1 - r_1)}, \quad \phi = 2\pi r_2$$

$$z_1 = r\cos\phi, \quad z_2 = r\sin\phi$$

Para μ, σ: $x = \mu + \sigma z$.

```
double r1 = rng.Uniform(0,1);
double r2 = rng.Uniform(0,1);
double z1 = sqrt(-2*log(1-r1))*cos(2*M_PI*r2);
double z2 = sqrt(-2*log(1-r1))*sin(2*M_PI*r2);
double x = mu + sigma*z1;
```

Monte Carlo Hit-or-Miss

Para amostrar $f(x)$ em $[a, b[$ com máximo m:

1. sorteie x uniforme em $[a, b[$
2. sorteie r uniforme em $[0, m[$
3. se $r < f(x)$, aceite x; senão, repita.

Eficiência:

$$\epsilon = \frac{\int_a^b f(x)dx}{(b-a)m}$$

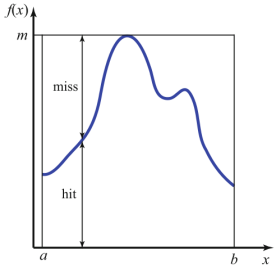


Figura: MC Hit and Miss

```
double x = rng.Uniform(a,b);
double r = rng.Uniform(0,m);
if (r < f(x)) return x; // aceito
```

Método da Aceitação-Rejeição

PyROOT — PDF definida pelo usuário:

```
import ROOT

# PDF definida pelo usuario (nao precisa ser normalizada)
def pdf(x):
    return ROOT.TMath.Exp(-x) * ROOT.TMath.Sin(x)**2

a, b, m = 0.0, 10.0, 1.0        # m >= max(pdf) em [a, b]
rng = ROOT.TRandom3(12345)

def sample(pdf, a, b, m):
    while True:
        x = rng.Uniform(a, b)
        y = rng.Uniform(0, m)
        if y < pdf(x):
            return x            # aceito

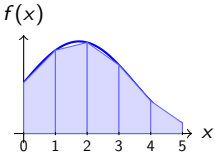
h = ROOT.TH1D("h", "Aceitacao-Rejeicao", 100, a, b)
for i in range(100000):
    h.Fill(sample(pdf, a, b, m))
```

Integração: Numérico × Monte Carlo

Queremos $I = \int_a^b f(x)\,dx$. Abordagens:

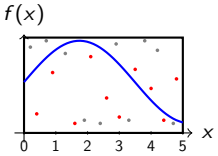
Numérico (trapézio):

$$I \approx \sum_{i=1}^{N} \frac{f(x_{i-1}) + f(x_i)}{2}\,\Delta x, \quad \text{erro } \mathcal{O}(1/N^2).$$



Monte Carlo (hit-or-miss):

$$I \approx \frac{N_{\text{hit}}}{N_{\text{tot}}}\,(b-a)\,m, \quad \text{erro } \propto 1/\sqrt{N}.$$



● hit ● miss

Ajuste de Sinal + Background

- Dados N eventos queremos estimar o número de eventos de sinal (s) e de background (b).
- O número de eventos N segue uma distribuição de Poisson:

$$P(N|b) = \frac{b^{N-b}}{N!}, P(N|s+b) = \frac{(s+b)^{N-(s+b)}}{N!},$$

  onde uma é a hipótese se ter somente background e outra de sinal+background.
- A observação destes eventos nos dá um vetor de medidas $x = (x_1, \ldots, x_N)$ e o vetor dos parâmetros $\theta = (\theta_1, \ldots, \theta_p)$.,
- Likelihood estendida:

$$L(x; s, b, \theta) = \frac{(s+b)^N e^{-(s+b)}}{N!} \prod_{i=1}^{N} \left(\frac{s}{s+b} f_s(x_i; \theta) + \frac{b}{s+b} f_b(x_i; \theta) \right)$$

$$\ln L(s, b, \theta) = -s - b + \sum_{i=1}^{N} \ln \left(\frac{s}{s+b} f_s(x_i; \theta) + \frac{b}{s+b} f_b(x_i; \theta) \right)$$

- Maximizamos $\ln L$ para obter $\hat{s}$, $\hat{b}$ e $\hat{\theta}$.

5. Modelo de sinal mais background

Para a massa invariante reconstruída, usamos um modelo com dois componentes:

$$f(m) = w_s f_s(m; m_H, \Gamma) + w_b f_b(m; \lambda), \qquad w_s + w_b = 1.$$

O sinal é descrito por uma distribuição de Breit–Wigner:

$$f_s(m; m_H, \Gamma) = \frac{1}{2\pi} \frac{\Gamma}{(m - m_H)^2 + \Gamma^2/4}.$$

Aqui:

- m_H é a massa do bóson de Higgs e Γ é a largura de decaimento;
- o máximo da distribuição ocorre em $m = m_H$;
- a largura a meia altura é igual a Γ.

Um background exponencial normalizado no intervalo $[m_{\min}, m_{\max}]$ é

$$f_b(m; \lambda) = \frac{\lambda e^{-\lambda(m - m_{\min})}}{1 - e^{-\lambda(m_{\max} - m_{\min})}}.$$

6. Leitura dos arquivos ROOT e ajuste com Minuit

O programa em C++ com Pythia8 salva os eventos em um arquivo ROOT. Por exemplo, a árvore `events` contém o ramo `mH`.

```
import ROOT
import numpy as np
from iminuit import Minuit

file = ROOT.TFile.Open("pythia_higgs.root")
tree = file.Get("events")

m = np.array([ev.mH for ev in tree])
mmin, mmax = 2.5, 3.5

def fs(x, mu, gamma):
    # Breit--Wigner normalizada em todo o eixo real
    return (1./(2.*np.pi))*gamma / (
        (x-mu)**2 + gamma**2/4.
    )

def fb(x, lam):
    norm = lam/(1.-np.exp(-lam*(mmax-mmin)))
    return norm*np.exp(-lam*(x-mmin))

def nll(mu, gamma, lam, s, b):
    pdf = s*fs(m,mu,gamma) + b*fb(m,lam)
    return s+b-np.sum(np.log(np.maximum(pdf,1.e-300)))
```

6. Leitura dos arquivos ROOT e ajuste com Minuit

```
fit = Minuit(nll, mu=3.10, gamma=0.04,
             lam=2.0, s=450., b=1560.)

fit.limits["gamma"] = (1.e-3, 0.3)
fit.limits["lam"] = (1.e-3, 20.)
fit.limits["s"] = (0., None)
fit.limits["b"] = (0., None)

fit.errordef = Minuit.LIKELIHOOD
fit.migrad()
fit.hesse()

print(fit.values)
print(fit.errors)
```

Os principais parâmetros do ajuste são

$$(m_H, \Gamma, \lambda, s, b).$$

7. Histogramas e fitting

O histograma mostra os dados simulados. A curva contínua representa

$$N_{\text{pred}}(m) = \Delta m \left[s f_s(m; m_H, \Gamma) + b f_b(m; \lambda) \right].$$

```
import matplotlib.pyplot as plt

bins = np.linspace(mmin, mmax, 101)
counts, edges = np.histogram(m, bins=bins)
centers = 0.5*(edges[1:] + edges[:-1])
errors = np.sqrt(np.maximum(counts, 1.))

x = np.linspace(mmin, mmax, 1000)
p = fit.values
dm = edges[1] - edges[0]

total = dm*(p["s"]*fs(x,p["mu"],p["gamma"])
          + p["b"]*fb(x,p["lam"]))

signal = dm*p["s"]*fs(x,p["mu"],p["gamma"])
background = dm*p["b"]*fb(x,p["lam"])

plt.errorbar(centers, counts, yerr=errors,
             fmt="ko", ms=3, label="dados")
plt.plot(x, total, "b-", lw=2, label="sinal + background")
plt.plot(x, signal, "r--", label="Breit--Wigner")
plt.plot(x, background, "g--", label="background")
```

7. Histogramas e fitting

```
plt.xlabel(r"$m\;(\mathrm{GeV})$")
plt.ylabel("eventos por bin")
plt.legend()
plt.tight_layout()
plt.show()
```

A posição do pico fornece $\hat{m}_H$ e sua largura fornece uma estimativa de $\hat{\Gamma}$.

9. Por que minimizar χ^2?

O valor de χ^2 mede a discrepância entre os dados e a previsão:

$$\chi^2 = \sum_i \left(\frac{\text{observado}_i - \text{esperado}_i}{\text{incerteza}_i} \right)^2 .$$

Cada termo compara o desvio em unidades do erro estatístico.

- Se o modelo descreve bem os dados, os resíduos são pequenos e χ^2 tende a ser baixo.
- Um desvio de 2σ contribui aproximadamente com 4 para o valor de χ^2.
- Parâmetros que produzem uma curva muito distante dos dados geram valores grandes de χ^2.

Portanto, a estimativa de mínimos quadrados é

$$\hat{\theta} = \arg\min_{\theta} \chi^2(\theta).$$

O algoritmo `Minuit` procura numericamente esse ponto no espaço dos parâmetros.

10. Intervalos de 1σ e 2σ

Após o ajuste, seja

$$\theta = (m_H, \Gamma_H).$$

A aproximação gaussiana usa a matriz de covariância:

$$\chi^2(\theta) = \chi^2_{\min} + (\theta - \hat{\theta})^T V^{-1} (\theta - \hat{\theta}),$$

onde $\Delta\chi^2(\theta) = \chi^2(\theta) - \chi^2_{\min}$ e

$$V = \mathrm{Cov}(\hat{\theta}_1, \hat{\theta}_2) = \begin{pmatrix} \sigma^2_{\hat{\theta}_1} & \mathrm{Cov}(\theta_1, \theta_2) & \cdots \\ \mathrm{Cov}(\theta_2, \theta_1) & \sigma^2_{\hat{\theta}_2} & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}$$

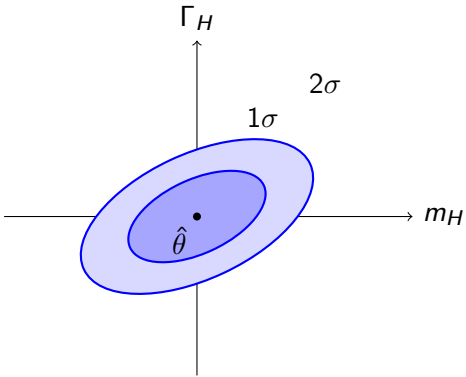
e

$$\left(V^{-1}\right)_{ij} = \frac{1}{2} \frac{\partial^2 \chi^2}{\partial\theta_i \, \partial\theta_j}\bigg|_{\theta=\hat{\theta}}$$

Para dois parâmetros:

$$\Delta\chi^2 = 2.30 \quad \text{para } 1\sigma, \qquad \Delta\chi^2 = 6.18 \quad \text{para } 2\sigma.$$

10. Intervalos de 1σ e 2σ



Conclusão

A massa do Higgs é extraída da posição do pico, enquanto a largura e os rendimentos são parâmetros adicionais determinados pelo ajuste.

8. Ajuste de qui-quadrado

Para dados binados, com contagens observadas n_i, comparamos os dados com a previsão do modelo $\nu_i(\theta)$:

$$\chi^2(\theta) = \sum_{i=1}^{N_{\text{bins}}} \frac{[n_i - \nu_i(\theta)]^2}{\sigma_i^2}.$$

Para contagens suficientemente grandes, podemos usar a aproximação Poisson-Gaussiana

$$\sigma_i^2 \simeq n_i.$$

O modelo de previsão para o bin i é

$$\nu_i(\theta) = \int_{m_i}^{m_{i+1}} [sf_s(m; m_H, \Gamma) + bf_b(m; \lambda)]\ dm.$$

Na prática, para bins estreitos,

$$\nu_i(\theta) \simeq \Delta m\, [sf_s(m_i; m_H, \Gamma) + bf_b(m_i; \lambda)]\,.$$

8. Ajuste de qui-quadrado

Para dados binados, pode-se minimizar

$$\chi^2(\theta) = \sum_{i=1}^{N_{\text{bins}}} \frac{[n_i - \nu_i(\theta)]^2}{\sigma_i^2}, \qquad \sigma_i^2 \simeq n_i.$$

```
def chi2(mu, gamma, lam, s, b):
    """
    Qui-quadrado para o modelo:

        sinal      = Breit-Wigner(mu, gamma)
        background = exponencial(lam)

    mu    : massa central da ressonancia
    gamma : largura Gamma da Breit-Wigner
    lam   : parametro do background
    s     : numero esperado de eventos de sinal
    b     : numero esperado de eventos de background
    """

    # Largura de cada bin
    bin_width = edges[1] - edges[0]
```

8. Ajuste de qui-quadrado

```
    # Previsao do modelo em cada centro de bin
    prediction = (
        s * fs(centers, mu, gamma)
        + b * fb(centers, lam)
    ) * bin_width

    # Variancia aproximada para contagens Poissonianas
    # O maximo com 1 evita divisao por zero
    variance = np.maximum(counts, 1.0)

    # Estatistica de qui-quadrado
    residuals = counts - prediction

    return np.sum(residuals**2 / variance)


# Ajuste dos parametros com Minuit
fit_chi2 = Minuit(
    chi2,
    mu=3.10,        # massa inicial
    gamma=0.04,     # largura inicial Gamma
    lam=2.0,        # parametro inicial do background
    s=450.0,        # rendimento inicial do sinal
    b=1560.0        # rendimento inicial do background
)
```

8. Ajuste de qui-quadrado

```
# Limites fisicos e numericos
fit_chi2.limits["mu"] = (mmin, mmax)
fit_chi2.limits["gamma"] = (1.e-3, 0.3)
fit_chi2.limits["lam"] = (1.e-3, 20.0)
fit_chi2.limits["s"] = (0.0, None)
fit_chi2.limits["b"] = (0.0, None)

# Para qui-quadrado, o erro-padrao eh definido por Delta chi^2 = 1
fit_chi2.errordef = Minuit.LEAST_SQUARES

# Minimizacao do qui-quadrado
fit_chi2.migrad()

# Estimativa das incertezas pela matriz de covariancia
fit_chi2.hesse()

# Resultados do ajuste
print("Parametros ajustados:")
print(f"m_H      = {fit_chi2.values['mu']:.5f} "
      f"+/- {fit_chi2.errors['mu']:.5f}")

print(f"Gamma    = {fit_chi2.values['gamma']:.5f} "
      f"+/- {fit_chi2.errors['gamma']:.5f}")

print(f"lambda   = {fit_chi2.values['lam']:.5f} "
      f"+/- {fit_chi2.errors['lam']:.5f}")

print(f"sinal    = {fit_chi2.values['s']:.2f} "
      f"+/- {fit_chi2.errors['s']:.2f}")
```

8. Ajuste de qui-quadrado

```
print(f"background = {fit_chi2.values['b']:.2f} "
      f"+/- {fit_chi2.errors['b']:.2f}")

print(f"chi2_min = {fit_chi2.fval:.2f}")
print(f"ndof     = {len(counts) - fit_chi2.nfit}")
print(f"chi2/ndof = "
      f"{fit_chi2.fval/(len(counts)-fit_chi2.nfit):.2f}")
```

- O ajuste de qui-quadrado depende da binning.
- O ajuste de máxima verossimilhança não agrupada utiliza cada evento individualmente.
- Para contagens pequenas, a verossimilhança de Poisson é preferível.